

# Cadena de P procesos con semaforos

Nuria Guerra Casal    Adrian Gijon Yubero

Grao en Enxeneria Informatica

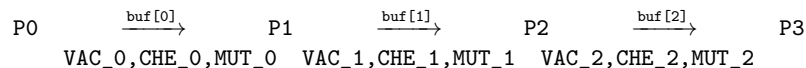
## 1. Introducción

Este ejercicio generaliza el problema productor-consumidor del apartado 2 a una **cadena de P procesos**. El primero de ellos actúa como productor exclusivamente: lee caracteres de un fichero de texto y los deposita en el primer buffer compartido. El último proceso actúa como consumidor exclusivo: extrae caracteres del último buffer y cuenta las vocales. Los procesos intermedios, si los hay, actúan simultaneamente como consumidores del buffer anterior y productores del siguiente, aplicando una **transformación** al carácter: calcula el siguiente carácter en el alfabeto antes de pasarlo al siguiente proceso ( $z \rightarrow a$ ,  $Z \rightarrow A$ , cualquier otra letra avanza una posición y los caracteres no alfabéticos pasan sin modificar).

El requisito principal es que los  $P$  procesos actúen **concurrentemente en la mayor medida posible**, lo que se logra al crearlos mediante `fork()` desde un proceso padre que previamente ha inicializado los recursos compartidos.

## 2. Estructura de la solucion

Para  $P$  procesos se crean  $P - 1$  buffers compartidos y  $3 \times (P - 1)$  semáforos con nombre, tres por cada buffer. Por ejemplo, la estructura de la cadena con  $P = 4$  sería la siguiente:



Cada proceso tiene un rol implementado como una función separada. El **proceso 0** (`rol_productor`) lee del fichero e inserta en `buf[0]`, exactamente igual que `prod_sem.c`. Los **procesos intermedios** (`rol_intermedio`) extraen del buffer de entrada, aplican `siguiente()` al carácter y lo insertan en el buffer de salida. El **proceso  $P - 1$**  (`rol_consumidor`) extrae del último buffer y cuenta vocales, igual que `cons_sem.c`.

Los buffers se alojan en un único `mmap()` de  $P - 1$  estructuras contiguas, de modo que todos los procesos tienen acceso a cualquier buffer con un simple índice. Cada buffer es idéntico al de los apartados obligatorios:

```
1 typedef struct {
2     char pila[TAMANO_BUFFER]; // pila LIFO de caracteres
3     int cima;                  // siguiente posicion libre
4 } DatosCompartidos;
```

Listing 1: Estructura de cada buffer de la cadena

## 3. Mecanismo de sincronizacion

Cada buffer esta protegido por el mismo triplete de semáforos que en el ejercicio 2. El semáforo `VAC_i` (valor inicial  $N = 10$ ) cuenta los huecos libres en el buffer  $i$  y bloquea al productor de ese enlace cuando está lleno. El semáforo `CHE_i` (valor inicial 0) cuenta los elementos disponibles y bloquea al consumidor de ese enlace cuando esta vacío. El semáforo `MUT_i` (valor inicial 1) es un mutex binario que garantiza que sólo un proceso a la vez accede al buffer  $i$ . Las funciones `insert_item()` y `remove_item()` son idénticas a las de `prod_sem.c` y `cons_sem.c`:

```

1 sem_wait(vacias); // espera hueco libre
2 sem_wait(mutex); // entra en exclusion mutua
3     buf->pila[buf->cima] = caracter;
4     buf->cima++;
5 sem_post(mutex); // sale de la region critica
6 sem_post(cheas); // avisa al siguiente proceso

```

Listing 2: Protocolo de insercion

El proceso padre crea **todos los semáforos antes de ninguna llamada a `fork()`**. De esta forma, cuando los procesos hijo empiezan, los semáforos ya existen y cada hijo los abre con `sem_open(..., 0)` sin necesidad de inicializarlos. Esto elimina la condición de carrera en la inicialización y garantiza que todos los procesos comienzan a trabajar de forma inmediata.

## 4. Concurrency

El mecanismo que garantiza la máxima concurrencia es la separación de los recursos: como cada par de procesos consecutivos comparte un buffer independiente con sus propios semáforos, el proceso 0 puede estar insertando en `buf[0]` mientras el proceso 1 lee de `buf[0]` y escribe en `buf[1]` y el proceso 2 lee de `buf[1]`, todo simultáneamente. Los semáforos solo bloquean cuando es estrictamente necesario (buffer lleno o vacío), no por ninguna sincronización global.

El lanzamiento mediante `fork()` asegura que los  $P$  procesos están activos casi al mismo tiempo. El padre no espera entre `fork()`s: lanza todos los hijos y luego llama a `waitpid()` sobre cada uno para esperar su finalización antes de liberar los recursos.

## 5. Velocidades y fases de ejecucion

Para observar el comportamiento del buffer, el proceso productor y el consumidor adaptan su velocidad en tres fases, al igual que en el apartado 2. Las **primeras 30 iteraciones** el productor no duerme y el consumidor duerme 3 segundos por iteración, de modo que el primer buffer se llena rápidamente y el productor se bloquea en `sem_wait(vacias)`. Las **siguientes 30 iteraciones** el productor duerme 3 segundos y el consumidor no duerme, vaciando el último buffer. Las **últimas 20 iteraciones** ambos duermen un tiempo aleatorio entre 0 y 3 segundos.

## 6. Ejecución

El programa se compila y ejecuta desde un único terminal:

```

# Compilar
gcc -o cadena cadena.c -pthread

# Ejecutar con P=4 procesos sobre un fichero de texto
./cadena 4 texto.txt

# Otro ejemplo
./cadena 2 texto.txt # equivalente a prod_sem + cons_sem (sin transformacion)

```

No es necesario abrir múltiples terminales: el programa lanza internamente los  $P$  procesos con `fork()` y espera a que todos terminen. Al finalizar, el proceso padre elimina todos los semáforos con `sem_unlink()` y libera la memoria compartida con `munmap()`.

## 7. Resultados y conclusiones

Ejecutando `./cadena 4 texto.txt` con el fichero “Este es un ejemplo productor-consumidor.” se observan varios fenómenos. En primer lugar, la transformación es visible en la salida: cada proceso intermedio imprime el carácter recibido y el transformado, por ejemplo `'e' → 'f' o 's' → 't'`. Con dos procesos intermedios ( $P = 4$ ), cada carácter avanza dos posiciones en el alfabeto antes de llegar al consumidor final.

```
[P0-Productor] Le 't' → buffer[0]
[P1-Intermedio] 't' → 'u' (buffer[0] → buffer[1])
[P2-Intermedio] 'u' → 'v' (buffer[1] → buffer[2])
[P1-Intermedio] 'u' → 'v' (buffer[0] → buffer[1])
[P3-Consumidor] Consume 'v'
```

Figura 1: Captura de ejecución del programa en terminal.

En segundo lugar, el **conteo de vocales del productor y del consumidor no coincide**, y esto es el comportamiento correcto y esperado: las vocales originales `a, e, i, o, u` se han transformado en `b, f, j, p, v` respectivamente, que son consonantes, por lo que el consumidor cuenta vocales distintas a las que leyó el productor. Esto demuestra que la cadena funciona correctamente.

```
[P0-Productor] Rematou. Vogais lidas: a=0 e=10 i=2 o=10 u=6
[P2-Intermedio] '.' → '.' (buffer[1] → buffer[2])
[P3-Consumidor] Consume '.'
[P1-Intermedio] Rematou.
[P2-Intermedio] Rematou.
[P3-Consumidor] Consume 'k'
[P3-Consumidor] Consume 'u'
[P3-Consumidor] Consume 'p'
[P3-Consumidor] Consume 'e'
[P3-Consumidor] Consume 't'
[P3-Consumidor] Consume '-'
[P3-Consumidor] Rematou. Vogais consumidas: a=0 e=4 i=0 o=4 u=6
```

Figura 2: Segunda captura de ejecución del programa en terminal.

En tercer lugar, **no se producen carreras críticas**: la salida es siempre consistente, el valor de `cima` nunca es negativo ni supera  $N$ , y ningún carácter aparece duplicado ni perdido dentro de cada enlace de la cadena.

La conclusión principal es que el mecanismo de semáforos **escala correctamente** a  $P$  procesos encadenados. Basta replicar el mismo triplete de semáforos por cada buffer y los procesos se sincronizan de forma autónoma en cada enlace, sin ninguna coordinación global. Esto demuestra la modularidad del patrón productor-consumidor: cada par de procesos adyacentes es independiente de los demás, lo que permite un nivel de concurrencia proporcional al número de buffers de la cadena.